# Instrumente și Tehnici de Bază în Informatică
# Curs 7

December 5, 2025

# API (Application Programming Interface)

- Contract, abstractizare care specifică modul în care interacționează mai multe componente software
- Numele, signatura funcțiilor, tipuri de date, semantica, etc.
- Nu expune implementarea funcționalităților; putem apela funcții dintr-o bibliotecă fără să ne intereseze cum sunt implementate
- Nu ne interesează dacă implementarea internă se modifică; cât timp API-ul rămâne neschimbat, nu avem de modificat codul apelant
- Evităm să scriem cod care depinde de aspectele interne (breaking the interface)
- Exemplu: bibliotecă pentru stringuri: folosim doar funcțiile expuse, nu ne interesează cum este reprezentat intern string-ul

# Funcții de sistem (syscall-uri)

- API pus la dispoziție de sistemul de operare prin care programele user pot cere kernel-ului resurse, să efectueze operații, etc.
- comanda `strace(1)` afișează toate apeluri de sistem care au loc în execuția unui program
- Sisteme UNIX (Linux, MacOS) – API POSIX
- Windows – API Win32 (interfață peste syscall-uri native, care include mai multe funcționalități, e.g. elemente GUI)

# Funcții de sistem

Sa considerăm câte un `hello world` scris în limbajele C, respectiv Rust:

- `hello.c`:

```
#include <stdio.h>

int main(){
    printf("Hello world\n");
}
```

- `hello.rs`:

```
fn main() {
    println!("Hello, world!");
}
```

# Funcții de sistem

- `$ strace ./hello-c`

```
munmap(0x7f2ae0eca000, 211007)          = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x6), ...}) = 0
brk(NULL)                               = 0x557ce696d000
brk(0x557ce698e000)                     = 0x557ce698e000
write(1, "Hello world\n", 12Hello world
)            = 12
exit_group(0)                           = ?
+++ exited with 0 +++
```

- `$ strace ./hello-rust`

```
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7f066fdd200
mprotect(0x7f066fdd2000, 4096, PROT_NONE) = 0
sigaltstack({ss_sp=0x7f066fdd3000, ss_flags=0, ss_size=8192}, NULL) = 0
write(1, "Hello, world!\n", 14Hello, world!
)         = 14
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NULL) = 0
munmap(0x7f066fdd2000, 12288)           = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

# Funcții de sistem

Câteva exemple de apeluri de sistem POSIX:

- Fișiere: `open(2)`, `read(2)`, `write(2)`
- Procese: `fork(2)` `waitpid(2)`, `getpid(2)`, `getppid(2)`
- Memorie: `mmap(2)`, `brk(2)`
- Permisiuni: `chmod(2)`, `chown(2)`
- etc.

# Funcții de sistem

Implementare cross-platform pentru `printf(3)` simplificat:

```
#ifdef _WIN32
    #include <windows.h>
#else
    #include <unistd.h>
    #include <fcntl.h>
#endif

void printf(const char *s) {
    int len = 0;
    while (s[len]) { len++; }

    #ifdef _WIN32
        HANDLE stdout_handle = GetStdHandle(STD_OUTPUT_HANDLE);
        DWORD written;
        WriteFile(stdout_handle, s, len, &written, NULL);
    #else
        write(STDOUT_FILENO, s, len);
    #endif
}
```

# ABI (Application Binary Interface)

- Similar cu API, dar pentru cod mașină
- Stabileste lucruri precum: convenția de apelare, alinierea tipurilor de date, utilizarea registrilor, denumirea simbolurilor, etc.
  **Exemplu:** Cât este `sizeof Struct`?

```
struct Struct {
    int x;
    char c;
};
```

# ABI (Application Binary Interface)

- Limbaje de sistem de nivel mai înalt (C++, Rust) nu au în mod implicit un ABI stabil, dar pot folosi explicit unul prin construcții de tip `extern C`:

```
extern "C" {
    int add(int x, int y) { return x + y; }
}

#[no_mangle]
pub extern "C" fn add(x: i32, y: i32) -> i32 { x + y; }
```

**Observație:** `extern C` limitează, în esență, signatura (nu și implementarea) funcției la cod C (interzice, e.g. în C++, overloading, clase, template-uri, etc.)

# ABI (Application Binary Interface)

```
#include <vector>
#include <numeric>
#include <ranges>

extern "C" {
    // C signature
    int sum_positive(int* arr, int len) {
        // crazy modern C++ 20 implementation
        auto vec = std::vector<int>(arr, arr + len);

        auto positive = vec | std::ranges::views::filter([](int x){
            return x > 0;
        });

        return std::accumulate(positive.begin(), positive.end(), 0);
    }
}
```

```
$ g++ -shared -fPIC -o libmylib.so mylib.cpp -std=c++20
$ gcc main.c -L. -lmylib -o main
$ LD_LIBRARY_PATH=. ./main
```

# FFI (Foreign Function Interface)

- Avem o implementare a unei biblioteci (de obicei în C)
- Vrem să îi accesăm funcționalitatea din alte limbaje, fără să o reimplementăm
- FFI: permite apelarea funcțiilor scrise în alt limbaj, printr-o interfață stabilită
- Ne rămâne doar să implementăm wrappere convenabile
- Alte exemple în ecosisteme specifice: JVM (interoperabilitate între Java, Kotlin, Scala, Clojure, etc), .NET (C#, F#, etc), iOS (Swift, Objective C)

# FFI (Foreign Function Interface)

Exemplu: apelarea unei funcții C din Python

`mylib.c`:

```
int add(int x, int y){
    return x + y;
}
```

`ffi.py`:

```
import ctypes

mylib = ctypes.CDLL("./libmylib.so")

add = mylib.add
add.argtypes = [ctypes.c_int, ctypes.c_int]
add.restype = ctypes.c_int

print(add(41, 27))
```